

Beca de Alimentos

Sistema de gestión de comedor universitario

CUTonalá · Universidad de Guadalajara

Versión	v1.0 — Prototipo inicial
Fecha	18 de March de 2026
Autor	Proyecto académico
Institución	CUTonalá, UdeG — Guadalajara, Jalisco
Estado	En diseño — pendiente levantamiento de requerimientos con cliente

Nota: Este documento refleja el diseño inicial basado en suposiciones del autor. Los requerimientos definitivos deberán validarse con el departamento de becas del CUTonalá antes de iniciar el desarrollo.

Contenido

1. Descripción del proyecto
2. Stack tecnológico
3. Roles del sistema
4. Arquitectura general
 - 4.1 Capa cliente
 - 4.2 Capa backend
 - 4.3 Capa de base de datos
5. Modelo de datos
 - 5.1 Entidades
 - 5.2 Relaciones
6. API REST — Endpoints
 - 6.1 Auth
 - 6.2 Usuarios
 - 6.3 Becas
 - 6.4 QR
 - 6.5 Transacciones
 - 6.6 Notificaciones
 - 6.7 Reportes
7. Diseño de pantallas (mockups)
 - 7.1 Vista estudiante
 - 7.2 Vista cajero
 - 7.3 Panel admin
8. Próximos pasos

1. Descripción del proyecto

El sistema **Beca de Alimentos CUTonalá** es una aplicación móvil diseñada para digitalizar el proceso de gestión de la beca de alimentos que ofrece el Centro Universitario de Tonalá (CUTonalá) de la Universidad de Guadalajara.

Actualmente, el proceso es manual: el becario recibe una hoja física con casillas que el cajero de la cafetería va sellando conforme se canjes las comidas. Este sistema busca reemplazar esa hoja con una tarjeta virtual que el estudiante lleva en su teléfono, mostrando un código QR que el cajero escanea para registrar el canje.

Objetivos principales

- Eliminar el uso de hojas físicas selladas.
- Dar al estudiante visibilidad en tiempo real de su saldo de comidas.
- Dar al cajero una herramienta rápida para registrar canjes mediante QR.
- Dar al admin control total sobre becarios, becas y reportes de uso.
- Generar alertas automáticas cuando el saldo de un becario es bajo.

2. Stack tecnológico

Capa	Tecnología	Justificación
Frontend móvil	Flutter (Dart)	Multiplataforma iOS y Android, rendimiento nativo, excelente soporte para QF
Backend	Spring Boot (Java)	Robusto, escalable, ideal para APIs REST institucionales
Base de datos	PostgreSQL	Relacional, datos estructurados, excelente integración con Spring via JPA/Hil
Autenticación	JWT (JSON Web Tokens)	Sin estado, seguro, compatible con los 3 roles del sistema
Infraestructura	Servidor local escuela	Red interna del CUTonalá (a definir con TI)
Comunicación	REST API / JSON	Estándar, ligero, fácil de consumir desde Flutter

3. Roles del sistema

Estudiante (EST)	Accede con su código UdeG. Puede ver su tarjeta de beca, el saldo de comidas disponibles, su historial de canjes, y generar su código QR personal para presentarlo en caja.
Cajero (CAJ)	Accede con sus credenciales. Escanea el QR del estudiante, visualiza su información y saldo, y registra el canje de una comida. Ve un historial de los canjes que ha procesado en el día.
Admin (ADM)	Acceso completo al sistema. Puede registrar y gestionar becarios, asignar becas por semestre, consultar el historial global de transacciones, y ver reportes y estadísticas de uso.

4. Arquitectura general

El sistema sigue una arquitectura cliente-servidor en tres capas bien separadas. Los clientes Flutter se comunican con el backend Spring Boot exclusivamente a través de una REST API con JSON. El backend gestiona toda la lógica de negocio y persiste los datos en PostgreSQL. Todo corre dentro de la red interna del CUTonalá.

4.1 Capa cliente

Aplicación Flutter que compila a iOS y Android nativos. Contiene tres vistas diferenciadas por rol: vista estudiante, vista cajero y panel admin. La comunicación con el backend se realiza mediante llamadas HTTP autenticadas con JWT.

4.2 Capa backend

API REST construida con Spring Boot. Organizada en módulos independientes:

Módulo	Tipo	Responsabilidad
auth	Lógica de negocio	Login, generación y validación de JWT, control de roles
estudiantes	Lógica de negocio	CRUD de becarios
becas	Lógica de negocio	Asignación y control de las 60 comidas por semestre
transacciones	Lógica de negocio	Registro de cada canje de comida
qr	Servicios de soporte	Generación de QR cifrado y validación al escanear
notificaciones	Servicios de soporte	Alertas automáticas por saldo bajo
reportes	Servicios de soporte	Estadísticas y reportes para el admin

4.3 Capa de base de datos

PostgreSQL con 4 tablas principales. Spring Boot se conecta a través de JPA/Hibernate, lo que permite mapear las entidades Java directamente al esquema relacional.

5. Modelo de datos

5.1 Entidades

USUARIO

Campo	Tipo	Descripción
id	uuid PK	Identificador único
nombre	string	Nombre(s) del usuario
apellido	string	Apellido(s) del usuario
codigo_udg	string	Código universitario UdeG
email	string	Correo institucional
password_hash	string	Contraseña encriptada
rol	enum	ESTUDIANTE CAJERO ADMIN
carrera	string	Carrera (solo estudiantes)
created_at	timestamp	Fecha de registro

BECA

Campo	Tipo	Descripción
id	uuid PK	Identificador único
usuario_id	uuid FK	Referencia al estudiante
semestre	string	Ej: 2025-B
comidas_totales	int	Total asignado (60)
comidas_usadas	int	Canjes realizados
fecha_inicio	date	Inicio del semestre
fecha_fin	date	Fin del semestre
estado	enum	ACTIVA AGOTADA VENCIDA

TRANSACCION

Campo	Tipo	Descripción
id	uuid PK	Identificador único
beca_id	uuid FK	Beca que se descontó
cajero_id	uuid FK	Cajero que registró el canje
fecha_hora	timestamp	Momento exacto del canje
nota	string	Nota opcional del cajero

NOTIFICACION

Campo	Tipo	Descripción
id	uuid PK	Identificador único
usuario_id	uuid FK	Destinatario de la notificación
mensaje	string	Texto de la alerta
leida	boolean	Si el usuario ya la vio
fecha	timestamp	Momento de generación

5.2 Relaciones

Entidad origen	Cardinalidad	Entidad destino	Descripción
USUARIO	1 → N	BECA	Un estudiante puede tener una beca por semestre
BECA	1 → N	TRANSACCION	Una beca genera múltiples transacciones de canje
USUARIO (cajero)	1 → N	TRANSACCION	Un cajero registra múltiples transacciones
USUARIO	1 → N	NOTIFICACION	Un usuario puede recibir múltiples notificaciones

6. API REST — Endpoints

Todos los endpoints usan el prefijo base `/api/v1/`. La autenticación se realiza enviando el JWT en el header `Authorization: Bearer <token>`. Los roles indican quién tiene permiso de llamar cada endpoint.

6.1 Auth

Método	Endpoint	Descripción	Roles
POST	<code>/auth/login</code>	Autenticación, devuelve JWT	EST CAJ ADM
POST	<code>/auth/logout</code>	Invalida el token activo	EST CAJ ADM
GET	<code>/auth/me</code>	Perfil del usuario autenticado	EST CAJ ADM

6.2 Usuarios

Método	Endpoint	Descripción	Roles
GET	<code>/usuarios</code>	Lista todos los usuarios	ADM
GET	<code>/usuarios/{id}</code>	Detalle de un usuario	ADM
POST	<code>/usuarios</code>	Registrar nuevo becario	ADM
PUT	<code>/usuarios/{id}</code>	Actualizar datos del usuario	ADM
DELETE	<code>/usuarios/{id}</code>	Eliminar usuario	ADM

6.3 Becas

Método	Endpoint	Descripción	Roles
GET	<code>/becas</code>	Lista todas las becas	ADM
GET	<code>/becas/mia</code>	Beca activa del estudiante autenticado	EST
GET	<code>/becas/{id}</code>	Detalle de una beca	ADM
POST	<code>/becas</code>	Asignar beca a un estudiante	ADM
PUT	<code>/becas/{id}</code>	Actualizar datos de beca	ADM

6.4 QR

Método	Endpoint	Descripción	Roles
GET	/qr/mio	Genera el QR del estudiante autenticado	EST
POST	/qr/validar	Valida un QR escaneado y retorna datos del estudiante	CAJ

6.5 Transacciones

Método	Endpoint	Descripción	Roles
POST	/transacciones	Registrar canje de comida (descuenta saldo)	CAJ
GET	/transacciones/mias	Historial de canjes del estudiante autenticado	EST
GET	/transacciones	Historial global de canjes	ADM
GET	/transacciones/{id}	Detalle de una transacción	ADM

6.6 Notificaciones

Método	Endpoint	Descripción	Roles
GET	/notificaciones	Notificaciones del usuario autenticado	EST
PUT	/notificaciones/{id}/leer	Marcar notificación como leída	EST

6.7 Reportes

Método	Endpoint	Descripción	Roles
GET	/reportes/uso-semester	Comidas usadas agrupadas por semestre	ADM
GET	/reportes/uso-diario	Canjes agrupados por día	ADM
GET	/reportes/estudiantes-sin-saldo	Becarios con comidas agotadas	ADM

7. Diseño de pantallas (mockups)

Las pantallas siguen la identidad institucional del CUTonalá con azul universitario (#003087) y oro (#F5A800) como colores primarios. El diseño es amigable y colorido, optimizado para uso móvil (iOS y Android).

7.1 Vista estudiante

- Encabezado azul con nombre, código UdeG y avatar con iniciales.
- Tarjetas de métricas: comidas disponibles (azul) y usadas (ámbar).
- Cuadrícula de 60 sellos (10×6): azul oscuro = usada, azul claro = disponible.
- Barra de progreso del semestre con conteo exacto.
- Botón dorado prominente para mostrar el código QR.
- Historial de los últimos canjes con fecha y hora.

7.2 Vista cajero

- Encabezado azul con indicador de rol (Cajero · CUTonalá).
- Área de escaneo de QR con ícono visual y texto de instrucción.
- Tras escanear: tarjeta del estudiante con nombre, código y carrera.
- Resumen de saldo en verde: comidas disponibles y usadas/totales.
- Cuadrícula de sellos para referencia visual del cajero.
- Botón azul "Registrar comida" y botón gris "Cancelar".
- Lista de canjes recientes procesados por el cajero en el turno.

7.3 Panel admin

- Encabezado con nombre del sistema y semestre activo.
- Barra de navegación: Becarios / Transacciones / Reportes.
- Métricas rápidas: becarios activos, canjes del día, saldo bajo, agotadas.
- Tabla de becarios con nombre, código, carrera, comidas usadas y estado (activa/saldo bajo/agotada).
- Botón "+ Agregar becario" para registrar nuevos beneficiarios.
- Gráfica de barras de canjes por día de la semana.

8. Próximos pasos

Inmediato

- Agendar reunión de levantamiento de requerimientos con el departamento de becas del CUTonalá.
- Validar supuestos: número exacto de comidas, flujo del cajero, integración con sistemas existentes.
- Definir infraestructura de servidor con el área de TI de la universidad.

Corto plazo (desarrollo)

- Configurar proyecto Spring Boot con PostgreSQL y entidades JPA.
- Implementar módulo de autenticación con JWT y los 3 roles.
- Desarrollar endpoints por módulo: usuarios → becas → transacciones → QR.
- Montar proyecto Flutter y conectarlo a la API REST.
- Implementar el escáner de QR en la vista del cajero.

Mediano plazo

- Pruebas de integración del flujo completo (escaneo → canje → notificación).
- Piloto con un grupo reducido de becarios del CUTonalá.
- Ajustes post-piloto y despliegue en servidor de la escuela.